

LAB 1

Credit Default Risk Scoring API

Turning a handed-over model into a service operations can run

Field	Value
Course	DDM501 — AI in DevOps, DataOps, MLOps
Session	3 (Lab 1)
Format	Team lab, 3–4 members

1. Overview

1.1 Scenario

You are the ML engineer at a consumer finance company in Vietnam. The data science team has just handed you a trained model that estimates the probability a credit card customer will miss their next payment. It works in their notebook.

Risk operations cannot use a notebook. They need a service the loan origination system can call, that answers in milliseconds, that behaves the same in staging as in production.

1.2 What you deliver

Deliverable	Description
REST API	FastAPI service with /health, /predict, /predict/batch and /model/info
Contract	Pydantic schemas that reject malformed input at the edge with a 422
Container	Dockerfile and Compose file that run the service anywhere
Test suite	pytest covering the happy path, model behaviour, validation and batching
Documentation	README plus the auto-generated Swagger docs

2. Background

2.1 The data

30,000 credit card customers, 23 features, one binary target. The schema follows the UCI Default of Credit Card Clients dataset (Yeh & Lien, 2009).

Column group	Meaning
LIMIT_BAL	Credit limit, in NT dollars
SEX, EDUCATION, MARRIAGE, AGE	Demographics
PAY_0, PAY_2...PAY_6	Repayment status for months t-1 to t-6. -2 = no consumption, -1 = paid in full, 0 = revolving credit, 1-8 = months of delay
BILL_AMT1...BILL_AMT6	Statement amount for months t-1 to t-6
PAY_AMT1...PAY_AMT6	Amount actually paid for months t-1 to t-6
default_payment_next_month	Target. 1 = defaults next month

2.3 From probability to decision

The model returns a probability. The service turns it into one of three underwriting actions using two thresholds:

```
probability < 0.30          -> LOW      -> APPROVE
0.30 <= probability < 0.60 -> MEDIUM -> REVIEW    (a human underwriter looks)
probability >= 0.60        -> HIGH     -> DECLINE
```

Both thresholds live in `app/config.py` and are read from environment variables.

3. Setup

```
unzip ddm501-lab1-starter.zip
cd ddm501-lab1-starter

python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

python scripts/train_model.py      # ~30 seconds
```

The training script prints ROC AUC of about 0.75 and writes `models/credit_model.joblib`. If it does, your environment is correct and you can start on the TODOs.

4. Tasks

5 files carry TODOs.

Task	File	TODOs	What you are building
1	<code>app/schemas.py</code>	4	The API contract: field bounds, closed value sets, custom validators
2	<code>app/model.py</code>	4	Loading the artifact, building the feature frame, the decision rule
3	<code>app/main.py</code>	2	The <code>/predict</code> and <code>/predict/batch</code> endpoints
4	<code>Dockerfile</code>	8	Layer order, non-root user, health check, entrypoint
5	<code>docker-compose.yml</code>	1	Ports, the model volume, environment configuration

Task 1 — The contract (`app/schemas.py`)

Define what a valid application looks like, precisely enough that the framework can enforce it.

1. Complete `CreditApplication`: `marriage`, `age`, `pay_status`, `bill_amt`, `pay_amt`. The first three fields are done as a worked example.
2. Add two field validators: `pay_status` values must be between `-2` and `8`, and no `pay_amt` may be negative.
3. Complete `PredictionResponse` with its six fields.

4. Complete HealthResponse with its three fields.

Task 2 — The model wrapper (app/model.py)

1. `_load_model`: `joblib.load` the bundle, take pipeline and metadata out of it, log clearly, and re-raise `FileNotFoundError`.
2. `to_frame`: flatten the grouped API payload into the 23 flat columns the model was trained on, in `FEATURE_COLUMNS` order.
3. `decide`: map a probability to a risk band and a decision. Mind the order of the comparisons.
4. `score`: score one application and assemble the full response dict.

Task 3 — The endpoints (app/main.py)

1. `/predict`: guard on the model being loaded, score, return a `PredictionResponse`.
2. `/predict/batch`: same guard, then one call to `score_batch` for the whole list.

Task 4 — The container (Dockerfile)

There are 8 TODOs, each with the reasoning in the file. The four that matter most:

- Copy requirements.txt and install BEFORE copying the source.
- Run as a non-root user. A container running as root that gets compromised is a host running as root.
- The health check must call `/health` and check that `model_loaded` is true.
- Bind `uvicorn` to `0.0.0.0`, not `127.0.0.1`.

Task 5 — Compose (docker-compose.yml)

One service, NOT in the image:

```
volumes:
  - ./models:/app/models:ro
```

5. Verifying your work

```
# 1. Tests
pytest tests/ -v --cov=app --cov-report=term-missing

# 2. Run it
uvicorn app.main:app --reload
curl http://localhost:8000/health

# 3. Score a good applicant
curl -X POST http://localhost:8000/predict \
  -H 'Content-Type: application/json' \
  -d '{"limit_bal":300000,"sex":2,"education":1,"marriage":2,"age":38,
    "pay_status":[-1,-1,-1,-1,-1,-1],
    "bill_amt":[12000,11500,11000,10500,10000,9500],
```

```
"pay_amt": [12000, 11500, 11000, 10500, 10000, 9500]]'

# 4. Container
docker compose up --build
docker inspect --format='{{.State.Health.Status}}' credit-risk-api
```

6. Deliverables

6.1 What to submit

One GitHub repository link per team, containing the completed starter with your commit history.

- Working API: /health, /predict, /predict/batch, /model/info
- Completed Pydantic schemas with the two custom validators
- Dockerfile and docker-compose.yml that build and run
- README updated with anything specific to your implementation